Суперкомпьютерное моделирование и технологии

Задание 1, вариант 18. Этап 1: OpenMP-программа

Хроян Георгий Овикович, группа 609

Октябрь 2026

1. Постановка задачи

В области

$$D = \{(x, y) : -1 < x, y < 1\} \setminus \{(x-1)^2 + (y-1)^2 \le 1\}$$

(квадрат без четверти круга с центром в $(1, 1)$) требуется приближённо решить задачу Дирихле

$$-\Delta u = 1, \quad (x, y) \in D, \qquad u = \sin(x + y), \quad (x, y) \in \gamma = \partial D.$$

Метод фиктивных областей. Задача заменяется задачей в квадрате $\Pi = (-1, 1)^2$:

$$-\Delta v + Q(x, y)\,(v - v_0) = F(x, y), \qquad v\big|_{\partial\Pi} = v_0, \qquad v_0 = \sin(x + y),$$

где $Q = 0$, $F = 1$ в D и $Q = 1/\varepsilon^2$, $F = 0$ в фиктивной области $\hat{D} = \Pi \setminus \overline{D}$; $\varepsilon = h^2$, $h = \max(h_1, h_2)$. В $\hat{D}$ большой коэффициент Q прижимает решение к v_0, и граничное условие на дуге выполняется приближённо без явного задания границы.

Разностная схема. Сетка $x_i = -1 + ih_1$, $y_j = -1 + jh_2$, $h_1 = 2/M$, $h_2 = 2/N$. Во внутренних узлах

$$-w_{\bar{x}x,ij} - w_{\bar{y}y,ij} + Q_{ij} w_{ij} = F_{ij} + Q_{ij} v_0(x_i, y_j),$$

на границе квадрата $w_{ij} = v_0(x_i, y_j)$. Коэффициенты усредняются по клетке $\Pi_{ij} = [x_i \pm h_1/2] \times [y_j \pm h_2/2]$: при доле площади клетки внутри D, равной $s_{ij} = \text{mes}(\Pi_{ij} \cap D)/(h_1 h_2)$, получаем $F_{ij} = s_{ij}$, $Q_{ij} = (1 - s_{ij})/\varepsilon^2$. Площадь пересечения клетки с кругом вычисляется точно (через первообразную $\sqrt{R^2 - x^2}$ и включение-исключение по углам клетки).

Метод решения. Метод сопряжённых градиентов с диагональным предобуславливателем $D_{ij} = 2/h_1^2 + 2/h_2^2 + Q_{ij}$, начальное приближение $w^{(0)} = 0$, скалярное произведение $(u, v) = \sum h_1 h_2 u_{ij} v_{ij}$, остановка при $\|w^{(k+1)} - w^{(k)}\|_E < \delta = 10^{-6}$, не более $(M-1)(N-1)$ итераций.

2. Последовательная программа

Сеточные функции хранятся одномерными массивами размера $(M+1)(N+1)$, узел (i, j) имеет индекс $i(N+1)+j$. Граничные узлы содержат v_0 и не меняются; невязка, направление и Ap на границе равны нулю. Одна итерация CG — три прохода по сетке: вычисление Ap вместе с (Ap, p) и (p, p); обновление w, r вместе с новым (z, r); новое направление $p = z + \beta p$.

Последовательная версия прошла четыре попытки на Polus (каталог `01_serial/runs`):

В попытке 3 на Polus программа оказалась в 12 раз медленнее, чем на ноутбуке, при той же скорости на ноутбуке у прошлой версии кода. С помощью ИИ-ассистента (Claude) была выдвинута гипотеза о конфликтах в кэше: пять массивов, выделенных отдельно, имели

Таблица 1. История последовательной версии на Polus.

№	Изменение	Итерации, 80×80	Время, с, 800×1200
1	без предобуславливателя, Q и F по узлу	305	—
2	+ диагональный предобуславливатель	168	—
3	+ точная площадь клетки (совпадение с эталоном)	167	29.98
4	+ один блок памяти со сдвигом, готовая диагональ	167	17.74

одинаковое выравнивание, и элементы с одним индексом попадали в одни наборы кэша L1 (64 КБ) ядра POWER8. Размещение массивов в одном блоке со сдвигом 13 КБ между ними и хранение готовой диагонали ускорили программу в 1.7 раза.

3. OpenMP-программа

Пять циклов по узлам сетки (подготовка коэффициентов, старт CG и три цикла итерации) размечены директивой `parallel for`: строки i делятся между нитями поровну. Скалярные произведения считаются через `reduction`, каждая нить копит свою частичную сумму:

```
#pragma omp parallel for reduction(+:pAp, pp)
for (int i = 1; i < M; i++)
    for (int j = 1; j < N; j++) {
        int k = i * s + j;
        Ap[k] = D[k] * p[k] - ih1 * (p[k - s] + p[k + s]) - ih2 * (p[k - 1] + p[k + 1]);
        pAp += Ap[k] * p[k];
        pp += p[k] * p[k];
    }
```

Итерации CG выполняются последовательно; α, β и проверка остановки вычисляются одной нитью между параллельными циклами. Нити привязаны к ядрам переменной окружения `GOMP_CPU_AFFINITY` (GCC 4.8.5 на Polus не поддерживает `OMP_PLACES`): каждая нить — на первый аппаратный поток своего ядра; задача занимает узел целиком (`#BSUB -x`, 16 ядер POWER8). При 32 нитях — по две нити на ядро.

История попыток (каталог `02_openmp/runs`):

1. `parallel for` без `reduction`: гонка данных на суммах, на 4 и 16 нитях метод не сходится (лимит 6241 итерация, $w(0,0)$ до $-9 \cdot 10^8$).
2. `reduction`: решение верное, но на 400×600 ускорение лишь $\times 4.9$ на 16 нитях.
3. привязка нитей к ядрам и эксклюзивный узел: $\times 12.5$ на 16 нитях (рис. 1).

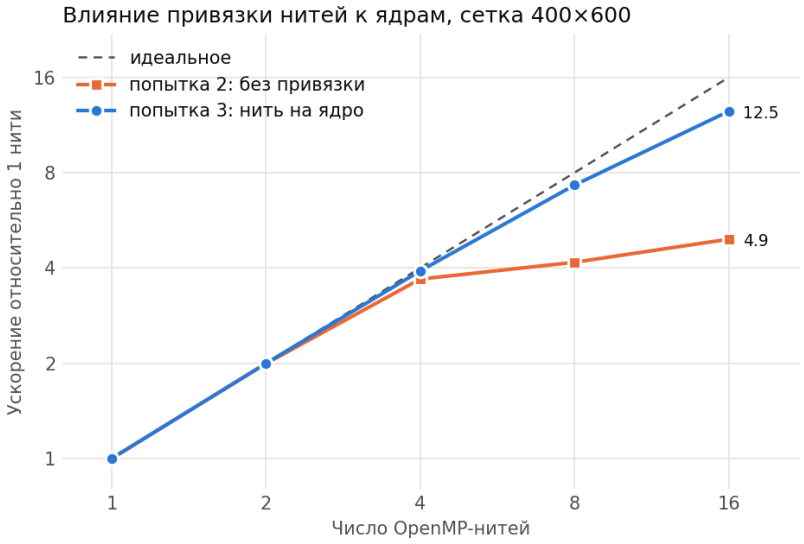


Рис. 1. Ускорение на сетке 400×600 без привязки нитей (попытка 2) и с привязкой (попытка 3).

4. Результаты

Все расчёты выполнены на IBM Polus в одном задании на эксклюзивном узле, каждая конфигурация дважды, в таблицах — минимальное время. Ускорение $S = T_{\text{serial}}/T_{\text{OpenMP}}$ на той же сетке.

Проверка на сетке 80×80 (табл. 2).

Таблица 2. OpenMP против последовательной программы, сетка 80×80.

Нити	Итерации	Время, с	Ускорение	$w(0,0)$	$\lvert w - w_{\text{serial}}\rvert$
1	167	0.0109	1.01	0.241097385417	0
4	167	0.0043	2.58	0.241097385417	0
16	167	0.0067	1.63	0.241097385417	0

Решение OpenMP-программы совпадает с последовательным на любом числе нитей. На сетке 80×80 мало работы, и накладные расходы на запуск и синхронизацию нитей сравнимы с вычислениями, поэтому 16 нитей медленнее 4.

Время последовательной программы, относительно которого считается ускорение, приведено в табл. 3, результаты OpenMP-программы — в табл. 4 и на рис. 2.

Таблица 3. Последовательная программа (база для ускорения).

Сетка $M \times N$	Итерации	Время, с	$w(0,0)$
80×80	167	0.0110	0.241097385
400×600	962	2.3279	0.243036155
800×1200	1824	17.6512	0.243249324

Сравнение сеток (рис. 3–5). На грубых сетках видна ступенчатая аппроксимация дуги γ клетками сетки; при сгущении граница сглаживается, а значение $w(0,0)$ устанавливается около 0.2432. Немонотонность на грубых сетках объясняется тем, что ошибка метода фиктивных областей зависит от того, как дуга проходит через клетки, и не убывает строго как $O(h^2)$.

Таблица 4. Результаты расчётов на ПВС IBM Polus (OpenMP-код).

Нити	Сетка $M \times N$	Итерации	Время, с	Ускорение
2	400×600	962	1.1328	2.05
4	400×600	962	0.5775	4.03
8	400×600	962	0.3100	7.51
16	400×600	962	0.1805	12.90
32	400×600	962	0.1934	12.04
2	800×1200	1824	8.5839	2.06
4	800×1200	1824	4.3298	4.08
8	800×1200	1824	2.2411	7.88
16	800×1200	1824	1.1806	14.95
32	800×1200	1824	1.0384	17.00

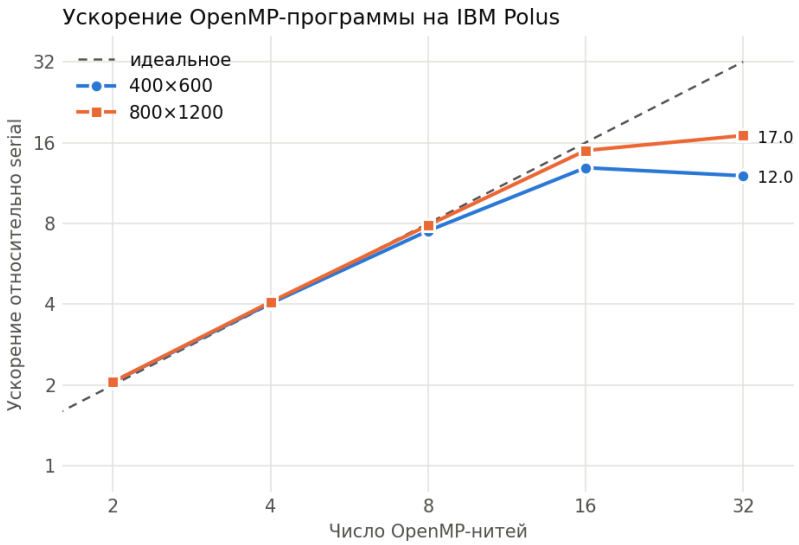


Рис. 2. Ускорение OpenMP-программы относительно последовательной.

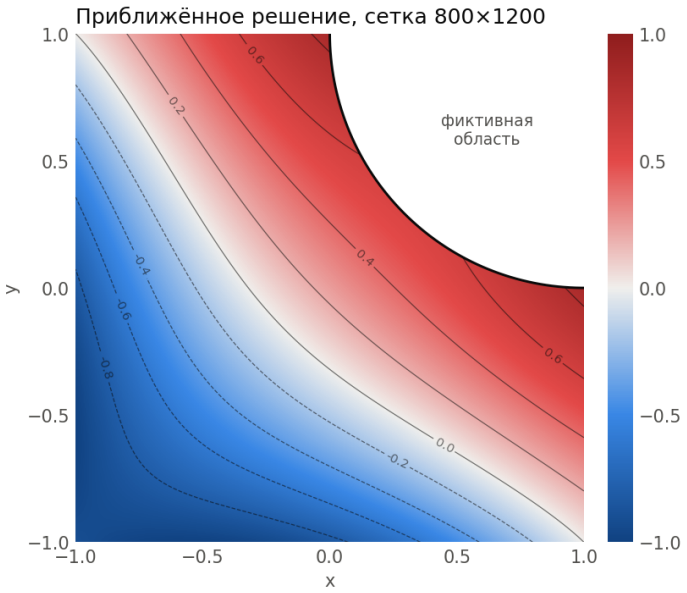


Рис. 3. Приближённое решение на сетке 800×1200; фиктивная область не показана, дуга — граница γ.

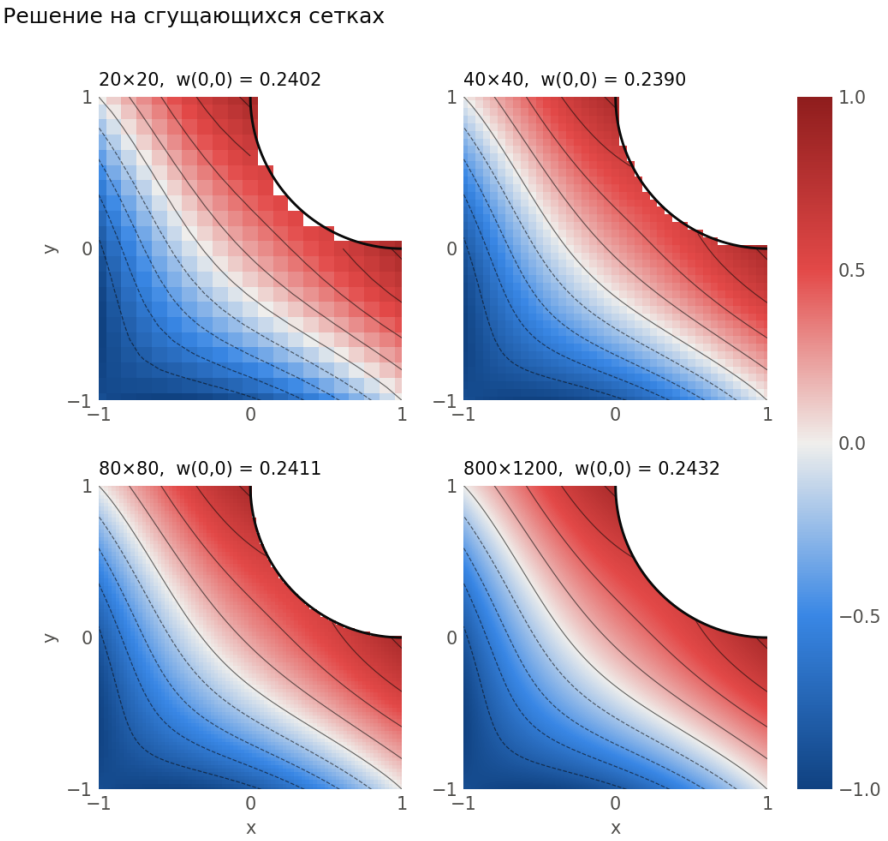


Рис. 4. Решение на сетках 20×20, 40×40, 80×80 и 800×1200 в одной шкале.

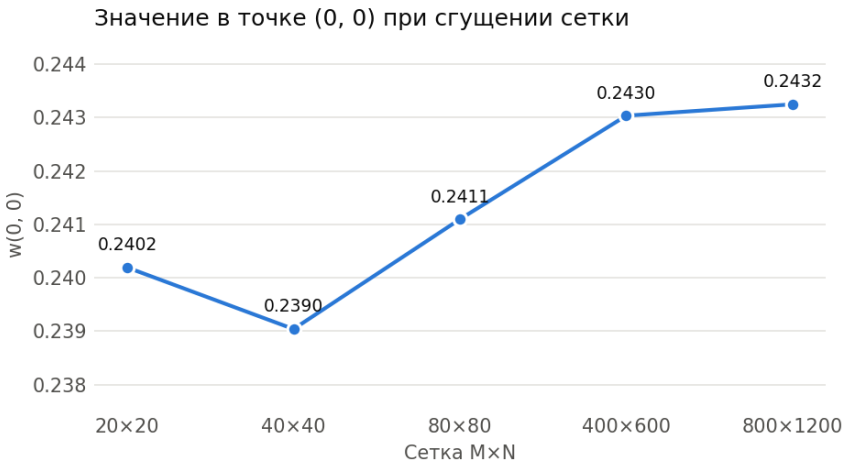


Рис. 5. Значение $w(0,0)$ на сгущающихся сетках (последовательная программа, Polus).

5. Выводы

- Последовательная и OpenMP-программы дают одинаковое решение; на сгущающихся сетках $w(0,0)$ устанавливается около 0.2432.
- До 16 нитей ускорение близко к линейному; на большей сетке оно выше, так как доля накладных расходов меньше.
- Две нити на ядро (32 нити на 16 ядрах) на сетке 800×1200 дают небольшой прирост ($\times 17.0$ против $\times 15.0$), а на 400×600 замедляют ($\times 12.0$ против $\times 12.9$): нити делят вычислительные блоки и кэш ядра, метод ограничен пропускной способностью памяти, а на меньшей сетке накладные расходы на синхронизацию 32 нитей перевешивают.
- Решающим для масштабирования оказалась привязка нитей к ядрам: без неё на 16 нитях ускорение было в 2.5 раза ниже.